

Class 2 Revision Notes — Anatomy of an LLM Call

Build, break, and observe an inference server · 25.07.2026

How to use these notes: Sections 1–9 cover what we did in class. Section 10 collects the Q&A discussions (worth revising — several exam-grade insights came from your questions). Sections 11–13 preview the break-the-server labs, which we'll run together in the extra lab class (poll coming: midweek or Sunday). The homework at the end is due two weeks from today.

1. What this class covered

Class 1 was the physics: you timed prefill and decode by hand in a notebook and watched a model think token by token. Class 2 is the engineering: the thing users actually hit is a **server**, and this class is about everything that happens between an HTTP request and a generated token.

In class we covered: the full anatomy of an LLM call (request, headers, streaming, response shapes), the client/gateway/engine mental model, the inference engine landscape, and the first lab — running TinyLlama under llama.cpp locally via Docker, plus Modal setup. The concurrency labs (breaking the naive server, comparing systems A/B/C/D) move to the **extra lab class**.

Recap from Class 1 (as discussed at the start)

- **Prefill** is computationally intensive, runs in parallel, and is a single pass over the whole prompt — compute-bound.
- **Decode** generates one token at a time and constantly re-reads model weights and the KV cache — memory-bandwidth-bound.
- **Memory limits concurrency.** The amount of memory available caps how many requests you can serve at once — this becomes the star of the break-the-server lab.
- Frontier models price **input vs output tokens differently** because prefill and decode have different costs. (Correction from class: the table said “T4 TPU” — it is a **T4 GPU**; and two students correctly caught a TTFT/sec computation bug in the Class 1 homework. The ratios in the cost table remain valid.)

2. The core mental model: three things that are not the same

People constantly confuse these three layers — in architecture reviews, incident channels, and interviews. Everything in this course hangs off this table.

Layer	What it does	What it does NOT do	Today's examples
Client	Builds the request, decides streaming, attaches headers	Does not run the model	curl, Open WebUI
Gateway (inference server)	Contract, routing, policy, queue, metrics, backpressure, admission control	Does not do the inference	naive FastAPI, RelayServe, LiteLLM
Engine	Prefill, decode, KV cache, batching, token generation, scheduling	Does not care about HTTP or clients	Transformers, llama.cpp

Main lesson of the course: a gateway in front of a bad engine still dies. An engine without a gateway has no queue, no metrics, and no contract.

Terminology (asked in class)

“Inference server”, “inference gateway”, and “API gateway” are used **interchangeably** in industry — software people say API gateway, inference people say inference gateway or inference server. The **engine** is a distinct thing: it does the calculations. The **model** is distinct again: it is just the learned weights (the QKV parameter matrices saved after training). A 50B-parameter model has seen more data than a smaller one — that is all “model” means; the engine is the software that executes it.

A useful analogy from class: the gateway is the **parent/guardian** of the engine — a harness that decides which requests deserve the engine's attention. The engine attends to every request in the order it arrives; it has no idea about clients, priorities, or authorization. Someone must decide **before** the engine — that someone is the gateway. (Barclays spent ~6 months with an entire team building just their inference gateway, precisely so their engines never waste compute.)

Why the separation matters: the Grafana dashboard story

Companies show up with beautiful dashboards and say “our TTFT is 50 ms, competitors claim 10 ms — what are we doing wrong?” Usually they **cannot answer where the latency comes from** — application layer? network? inference engine? hardware? Without a clean layer separation, latency and throughput cannot be attributed, and un-attributable latency cannot be fixed. That attribution discipline is exactly what the A/B/C/D lab trains.

3. What actually happens when you make an LLM call

1. **Client constructs the request** — model, messages, max_tokens, stream, temperature. (When you call OpenAI, the SDK builds this for you behind the scenes.)
2. **Client attaches control & identity** — Authorization header + metadata headers.
3. **Request is transmitted** — HTTPS. (Not HTTP. There's Wireshark for people who get that joke.)
4. **GATEWAY validates, admits, routes** — and may queue you.
5. **ENGINE assembles context** → **prefill** → **decode** — token by token, shaped by temperature/top-p/stop.
6. **Response is delivered** — one JSON object, or streamed chunks.
7. **Post-request accounting** — tokens counted, latency logged, metrics emitted. Every engine ships different metrics (vLLM ≠ SGLang ≠ llama.cpp); you can always add your own on top.

Steps 4 and 5 are this class. Everything else exists to serve them.

4. The request: what the client sends

Field	What it does	Why you care
model_id	Selects architecture + capability tier	It effectively selects the PRICE
prompt / messages	Input context	Often dominates cost (remember input-token pricing)
max_tokens	Upper bound on generated output	The server RESERVES budget for it — see warning below
temperature / top_p	Randomness vs determinism	Behavioral, not structural
stream	Incremental token delivery	Changes the entire delivery pipeline (Section 6)

stop	Hard termination strings	Cost control
------	--------------------------	--------------

War story from class: people ship systems without a `max_tokens` limit — and users start sending entire books. No sensible deployment leaves `max_tokens` unset: it is your budget reservation and your protection.

Headers: control and identity

Every request carries two kinds of headers. They are **headers, not tokens** — they ride outside the payload.

- **Authorization headers** establish who is calling and what they may do — your API key / bearer token. When your gateway receives a request, it checks whether that role exists and is allowed; if the handshake fails, the request is discarded before it ever reaches the engine.
- **Metadata headers** attach everything else: request IDs (so a call can be traced across five services), tenant/user context, observability hooks. Look at Anthropic's and OpenAI's real headers — e.g. anthropic-version pins the API contract; request IDs come back so you can correlate logs.

The messages[] array and roles

Every message has a `role` and `content`. The server walks the array and decides per message: extract the prompt (role “user”), or ignore/discard. Design question hiding there: is an unknown role an error, or noise to skip?

Key insight: messages is an array because the CLIENT carries conversation history. The server is stateless — every request re-sends the whole conversation. That's why context (and cost) grows with every turn, and why “memory” in chat products is an illusion maintained by the client. Our naive server does the crudest thing possible: it grabs the last user message and ignores the rest — you'll read that code in the lab.

5. Responses: non-streaming vs streaming

Non-streaming

One JSON object: an `id` (your correlation handle), `choices[]` (usually one entry holding the assistant message), and `usage` (`prompt_tokens`, `completion_tokens`, `total_tokens`).

Never ignore usage. That object is your bill, your throughput metric, and your capacity signal. Even if you leave fields empty, define them — it was called out in class as one of the most common mistakes.

The shape looks boring — that is the point. This exact contract is what lets a browser UI, a curl command, and a Python SDK talk to the same server interchangeably. Boring contracts are what interoperability is made of.

Streaming

Same content as server-sent events: `data: {"delta": {"content": "Hello"}} ...` ending with the sentinel `data: [DONE]`. Until `[DONE]`, the request stays live in the system.

- The client must **assemble** the message by concatenating deltas.
- The arrival of the first delta is a measurable event: **TTFT** — time to first token. If you measure one thing, measure this.
- **The hard part:** if the connection dies at chunk 40 of 100 — who knows the request failed? With an engine hooked directly to your app, nobody keeps that record; the engine just stops. A gateway can record which request failed and decide: retry or flag. This ambiguity is what makes streaming genuinely hard for gateways — and few gateways handle it well.

Why streaming changes gateway design

For the gateway, streaming means: forward chunks **immediately** (every buffered millisecond is TTFT you're adding), **never buffer** the whole response, and handle **partial failure**. The gateway stops being a request/response forwarder and becomes part of the delivery pipeline.

6. REST vs gRPC (with the class debate)

	HTTP / REST	gRPC
Nature	Ubiquitous, firewall-friendly, JSON	Binary protocol over HTTP/2, Protobuf schemas
Typing	Loose	Strongly typed (schema-enforced)
Performance	Fine for most workloads	Lower latency, better throughput under heavy load
Streaming	Server-sent events (client←server)	Client, server, and bidirectional streams
Where you see it	Every consumer LLM API (POST /v1/chat/completions)	Internal microservices, high-QPS systems, banks (e.g. Bloomberg)

Class debate, recorded honestly: the slide's claim that gRPC is “not language-agnostic” was pushed back on — correctly. gRPC supports clients in many languages via Protobuf codegen, and it is not an obsolete protocol (it's ~10 years old, from Google, actively used). The fair statement: with gRPC you must **generate typed clients per language**, while REST needs nothing — any language can fire an HTTP request. Also noted from class: until SSE, REST was essentially client→server only, while gRPC always had bidirectional streaming.

Rule of thumb (Bloomberg, Cisco — confirmed by students in class): REST at the edge, gRPC inside. Users hit REST; your microservices talk gRPC to each other.

Side question from class — client-side tokenization: does anyone tokenize on the client? Very rare: tokenizers differ per model, so you'd have to ship many tokenizers with the client. Seen occasionally for older Hugging Face classification models in ONNX format. (For images/speech the calculus may differ — tokenization there is heavy — but that's outside LLM serving.)

7. Why any of this matters: inference is FinOps

Inference engineering is not a performance hobby — it is a **cost lever**. There is a dollar sign attached to every metric you'll measure: latency, throughput, hardware choice, concurrent requests. From the Class 1 cost table: for TinyLlama with a short prompt, ~86% of the cost was **output** tokens. People who come straight from computer engineering get this mental model wrong — at the end of the day, this is FinOps.

The three sources of every bottleneck (from class)

1. **Hardware** — what the silicon can do.
2. **Load** — how many requests you're accepting, i.e. how much pressure you put on that silicon.
3. **Optimization** — how well-engineered your pipelines are between the two.

This class and the next focus on the server side: how far can you push a system before it breaks — and it can break for several reasons, not just concurrency.

8. The inference engine landscape

All engines do the same core job — prefill, decode, KV cache — written by different people. They differ in model support, long-context handling, admission control, batching/queuing/scheduling features.

Engine	Notes from class
llama.cpp	C/C++, compiles on every device (Linux/Windows/Mac) — why we use it in this course. GGUF single-file quantized weights. Dominant on edge/IoT. Ships its own OpenAI-compatible HTTP server. Ecosystem now developing fast within Hugging Face.
vLLM	The most popular open-source serving engine; supports specific model families. The queue + FIFO scheduler + continuous batching diagram in the slides is vLLM's.
SGLang	Popular third step; works especially well for RAG workloads.
NVIDIA Triton / TensorRT-LLM	Enterprise path; kernel-level engineering and custom functions. What OpenAI uses. Best when fully inside the NVIDIA ecosystem.
NVIDIA Dynamo	Newer; several hyperscalers run it behind the scenes.
TGI (Hugging Face)	Earlier generation, less used now.
MLX	Apple's own engine for Apple silicon; Google similarly has TPU-focused stacks.

Typical adoption path (from class): raw Transformers → vLLM → SGLang → Dynamo / TensorRT-LLM as scale and NVIDIA-alignment grow. None is simply “best” — they trade model support, features, and ecosystem lock-in. A when-to-use-which list is coming in the course notes.

Cleared up in class: the KV cache lives in the ENGINE, not the model. The model is weights; the KV cache is runtime state the engine creates and manages per request. You don't “activate” it in the model — the engine implements it.

9. Lab walkthrough: what we ran (and why it's built this way)

Part 1 — inference_101.py: the engine as HTTP

What it does: hosts TinyLlama (GGUF, ~700 MB) behind llama.cpp's own server on port 8081, inside Docker, on your CPU. Then you speak to it with curl:

- `curl -s http://127.0.0.1:8081/health | jq` — GET = “are you alive?”
- `curl -s http://127.0.0.1:8081/v1/chat/completions -d '{...}' | jq` — POST = ChatGPT-shaped JSON from a process on your own laptop. Change the content field to anything — a longer prompt, an agent's context, RAG output — the request structure stays the same.

llama.cpp prints its own metrics per request (timings, cache, ms/token, predicted tokens/sec). We didn't write those — the engine ships them. They're the bare minimum: nothing yet about application, network, or GPU performance. We build the rest on top, class by class (Grafana dashboards are pre-wired in the repo's extras for a later class).

Why Docker instead of building llama.cpp?

llama.cpp is C/C++ built with CMake — and on Macs, CMake + Xcode gets temperamental. So we pull the prebuilt image from the GitHub container registry and run it in Docker. **General rule from class: never build**

an inference engine from source unless you're contributing to it. (Note: the image tag mentions CUDA but we're running CPU-only today; GPU profiling comes later.)

Why both a venv AND Docker? (asked in class)

The venv holds the Python-side requirements — FastAPI, clients, scripts that run on **your** machine. Docker holds the engine — the prebuilt llama.cpp binary and its runtime. Two different layers, two different packaging worlds. (This confusion is natural; it maps exactly to gateway-side vs engine-side.)

Modal setup (for the lab class)

1. Claim credits: the compute-credits form in Class 1 resources → 500 credits (check spam/promotions for the email).
2. Create/choose a workspace (make a separate one for the course if you use Modal at work).
3. Settings → API tokens → create token → copy **both** the token ID and token secret (shown together when created).
4. Put them in the env config (`.env` per `commands.md`). An `HF_TOKEN` is pre-wired for later model switching — for TinyLlama you don't need extra access.
5. This is authorization for pushing apps from your device — same idea as an SSH key for GitHub.

Troubleshooting seen live in class

Symptom	Fix
zsh: parse error near '>'	You pasted a URL wrapped in <angle brackets>. Remove them — always.
Cannot connect to the Docker daemon	Docker Desktop must be open as an application; keep it running all class.
modal token ID missing	You ran a Modal step early — for the local lab only commands 1–6 and 9 are needed; Modal env comes later.
pip installing mystery packages	Dependencies of requirements — let it finish. When versions fight, ask Cursor; life is short.
Windows command differences	Commands are written on/for Mac — ask Cursor for the Windows equivalent.

What Modal shows you (demoed live)

Modal's dashboards (deployment history, usage — T4 vs CPU, app logs, metrics) are mostly **re-surfaced engine metrics** with thin dashboarding — bare minimum, not complete information. That's not an insult: platforms like Modal, Lambda, and the neoclouds are excellent at one small thing and thin everywhere else — e.g. under concurrent load they won't queue or retry for you; your application (gateway!) must. What you're learning here is how to build what they built — starting from your own device and asking: what did they custom-engineer, and what would I add?

10. Q&A highlights (revise these — they were the best part)

Agentic workloads are CPU-bound

Everything about execution — which tool to call, routing, orchestrating workflows, planning — runs on the **CPU**. Only the forward pass (from tokenization onward) runs on the GPU. In agentic systems the model calls barely dominate; most time goes to tool-calling and orchestration. Hence the industry line: **agentic workloads are CPU-bound**. If an agent opens many parallel threads (hundreds of sources, sites, docs), CPU memory itself becomes the bottleneck — much more so than in RAG apps.

The hardware response: disaggregated prefill & faster CPU–GPU links

Two things are happening on the hardware side because everyone wants faster agents: (1) **disaggregated prefill** — run prefill on one device and decode on another, freeing decode capacity (this shifts some latency to the transfer between them, so it pays off when the lag is on the prefill side); (2) **faster CPU↔GPU and GPU↔GPU communication** — the evolution PCIe → NVLink → C2C, and the Vera Rubin architecture specifically targets CPU–GPU latency. On older systems it's a shared bus. The Cerebras/etched/AMD press releases discussed in class are about exactly this space.

Is RAG prefill-bound or decode-bound?

Worked out live in class: RAG ingests whole documents (huge prefill) and typically produces summaries (short decode) — so RAG is **usually prefill-bound**: TTFT is large, but inter-token latency is fine once generation starts. The honest caveat a student raised: it depends on the output — ask a RAG system for a long essay and decode grows. And agents that execute tools are bound elsewhere entirely (CPU). Classify workloads by **where the tokens are**, not by the product name.

Code generation

Decode-heavy, creative-class workload: prompts are modest, generated token counts are high — so stream the response rather than waiting for five files to finish. How does Cursor handle dependencies? Models are trained on GitHub-scale code (structure is in the weights), and agents **plan first, then execute** — dependency coherence is established in planning, then code is generated class by class, file by file, not all at once.

The orchestration layer (above the engine)

One GPU isn't enough → you run several engine copies → someone must orchestrate them. **Kubernetes** is the bare-bones layer (nodes, pods; you pick worker nodes with the CPU/GPU/VRAM your workload needs — the model, the engine, and your app deploy as separate pods and get chained). **Ray** is an abstraction on top (born a few years ago, strong on conventional ML). **LLM-d** is a new abstraction on top of Kubernetes optimized specifically for vLLM backends and very large MoE models (~300B+); real commits started Nov last year — it's that new. **Slurm** still runs under plenty of it, whatever names get invented. Expect five more LLM-x projects in the next two years.

Frontier problems (where this field is going)

- **Control plane**: consensus while generating (majority polling vs KV-metadata vs gossip), heartbeat/failure detection latency — orchestrator-side problems nobody has fully solved.
- **Compiler/kernels**: engines are moving down the stack — graph breaks (Triton/Dynamo teams), recompilation storms when bucket shapes change, JIT-at-serve-time vs AOT kernel caches, serve-time vs bench-time gaps. These become the engine problems of the next 1–2 years.
- **Dispatch**: load imbalance, expert parallelism for MoE, co-located vs disaggregated systems, supporting new architectures (MLA).

Test-time compute (asked directly)

Think of it as the compute your system spends at serve time — and it can blow past predictions because of **retries**. Every retry raises the question: recompute from scratch, or resume from a checkpoint? Engines are growing mechanisms to checkpoint (e.g. saving CUDA graphs between processes — Prashant from Inferact

will guest-lecture on exactly this). OpenAI's early pain: dying requests recomputed from scratch until they had enough GPUs to absorb it.

Multimodal serving (asked)

The course focuses on LLM serving; image/speech pipelines structure requests differently (tokenization is much heavier). Selecting engines/infra for multimodal gets its own treatment later. Same for home-lab setups — a small dedicated segment was promised.

11. Coming up in the lab class: build it, break it, fix it

The repo you cloned contains four deployable systems. The experiment is one variable at a time:

System	Engine	Gateway	Expected under load	Who fixed it?
A — Naive	Transformers	none	OOM + exploding TTFT	—
B — Engine only	llama.cpp	none	Survives	Engine
C — RelayServe	llama.cpp (→B)	RelayServe	Survives + queue + metrics	Engine + Gateway
D — LiteLLM	Transformers (naive)	LiteLLM	Still fails like A	Gateway alone ≠ enough

Why the naive server will break (read `naive_server/server.py` before the lab)

- `MODEL_LOCK` serializes the GPU — but the server still accepts **unlimited** concurrent requests. Every accepted request holds memory while it waits.
- One `model.generate()` per request — no batching. The GPU works one sequence while others camp on their KV caches.
- API + engine in one process — no queue limit, no backpressure, no admission control. A server that cannot say no dies of hospitality.

Prediction exercise (commit before the lab): concurrency goes $4 \rightarrow 8 \rightarrow 16 \rightarrow 32$. What dies first — memory (OOM) or TTFT? Write it down. Both are defensible; that's the point.

Attribution rules (memorize — reflection Q2 is graded against this)

- **Engine owns:** KV cache management, continuous batching, tokens/sec, OOM resistance.
- **Gateway owns:** stable contract, routing + health checks, queue depth + admission control surface, metrics/request IDs.
- **A gateway in front of a broken engine does NOT fix OOM** — that is exactly what System D exists to prove.

12. Observability: what to alert on

Metric	Why it pages you
TTFT p95	User-facing latency — the real SLA number (p95, not the average — the average lies)

Queue depth	LEADING indicator — grows before latency explodes; alert here to act early
OOM / 503 rate	Capacity exhausted
Tokens / sec	Decode health under load — sags before errors appear
GPU / KV memory	The real OOM predictor

GPU utilization % alone tells you almost nothing. 100% busy is GOOD if latency holds (that's efficiency) — and the naive server dies at LOW utilization (memory-bound + serialized). Utilization is a cost metric, not a health metric.

Also remember the honest latency equation: **total latency = queue time + network RTT + cold start + GPU compute.** A remote GPU is not automatically faster — measure, never assume.

13. Key takeaways

1. Client, Gateway, and Engine are three different jobs. Confuse them and you ship the naive server.
2. The engine owns the hard performance problems — KV, batching, memory.
3. The gateway owns the operational problems — contract, queue, metrics, routing.
4. You cannot improve what you cannot see — instrumentation is not optional.
5. Always measure. Remote GPUs are not automatically faster — network, cold start, and queue time still exist.

One sentence to remember: Gateway owns the contract and the queue. Engine owns the KV cache and the batching. Confuse the two and you will recreate the break-the-server lab in production.

14. Homework, deliverables & logistics

Deliverables (due in two weeks; ~2 h of work after the lab class)

1. `/healthz` + one successful chat screenshot on **Modal GPU** (local Docker doesn't count).
2. Part 3 break table + `/metrics` screenshot taken **during** the concurrency sweep.
3. Comparison table A vs C: completed requests, errors, OOM, avg/p95 TTFT, tok/s — real numbers.
4. Four reflections (≤ 300 words each): what broke and why · llama.cpp vs RelayServe split · RelayServe in front of the naive engine = ? · what would you alert on and why. (Q3 is the trap — you now know the answer: same OOM. Say why.)

Grading: evidence 20 · break 30 · table + split 25 · reflection 25. Extra credit: LiteLLM demo screenshot + the “Implement this” appendix slides. Collaboration is fine — the goal is that you learn to implement this; look at classmates' submitted solutions if stuck.

Logistics

- **Extra lab class:** poll coming — midweek (Wed/Thu) or next Sunday. That's where we run the break-the-server and A/B/C/D labs together.
- **Also being added by request:** a transformer-architecture walkthrough class (follow one token through the full stack, QKV math included), whiteboarding/diagrams for the systems discussions, and a list of inference engines with when-to-use-which.
- **Discord:** link is in the student portal and in the 23rd-July email (check spam/promotions).
- **Credit issues:** use the credit-issues form in the portal; the team resolves within hours on weekdays (not weekends).

- **Papers in the notes (mooncake, vLLM, FlashAttention, etc.):** read the abstracts only — they exist to build intuition for why each system was built. Full reads are optional; a JAX core maintainer will guest-lecture on the TPU side later.
- **After any Modal work:** `bash scripts/modal.sh stop all` — idle deployments consume credits.